

Análisis de Algoritmos

Clase 11 — Quicksort: algoritmo y análisis de peor caso

Luis Alfredo Alvarado Rodríguez

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
ESCUELA DE CIENCIAS FÍSICAS Y MATEMÁTICAS

1 de septiembre de 2025

Sumario

- 1 Modelo de ordenación
- 2 Quicksort
- 3 Partición y correctitud
- 4 Peor caso

Comparaciones e intercambios

Planteamiento

Supongamos que se nos da un arreglo $x[1], \dots, x[n]$ de n números.

Comparaciones e intercambios

Planteamiento

Supongamos que se nos da un arreglo $x[1], \dots, x[n]$ de n números. Quisiéramos reordenarlos en orden no decreciente usando:

- (a) comparaciones entre dos claves;

Comparaciones e intercambios

Planteamiento

Supongamos que se nos da un arreglo $x[1], \dots, x[n]$ de n números. Quisiéramos reordenarlos en orden no decreciente usando:

- (a) comparaciones entre dos claves;
- (b) intercambios de posición (swaps).

Comparaciones e intercambios

Planteamiento

Supongamos que se nos da un arreglo $x[1], \dots, x[n]$ de n números. Quisiéramos reordenarlos en orden no decreciente usando:

- (a) comparaciones entre dos claves;
- (b) intercambios de posición (swaps).

Ejemplo

$\{9, 4, 7, 2, 1\} \longrightarrow \{1, 2, 4, 7, 9\}$

slowsort y su costo

Esquema

- (a) encontrar la mínima; intercambiarla con la primera;

slowsort y su costo

Esquema

- (a) encontrar la mínima; intercambiarla con la primera;
- (b) repetir con la segunda mínima; etc.

slowsort y su costo

Esquema

- (a) encontrar la mínima; intercambiarla con la primera;
- (b) repetir con la segunda mínima; etc.

Comparaciones

$$f_1(n) = \sum_{r=1}^{n-1} \sum_{j=r+1}^n 1 = \sum_{r=1}^{n-1} (n-r) = \frac{(n-1)n}{2}.$$

slowsort y su costo

Esquema

- (a) encontrar la mínima; intercambiarla con la primera;
- (b) repetir con la segunda mínima; etc.

Comparaciones

$$f_1(n) = \sum_{r=1}^{n-1} \sum_{j=r+1}^n 1 = \sum_{r=1}^{n-1} (n-r) = \frac{(n-1)n}{2}.$$

⇒ **Comparaciones:** $\Theta(n^2)$ (mejor = peor).

Intercambios promedio: $\Theta(n^2)$ sobre las $n!$ permutaciones.

$$26, 18, 4, 9, \mathbf{37}, 119, 220, 47, 74. \quad (2.1)$$

26, 18, 4, 9, **37**, 119, 220, 47, 74. (2.1)

- Si un elemento queda con todos los menores a la izquierda y los mayores a la derecha, basta ordenar cada lado.

26, 18, 4, 9, **37**, 119, 220, 47, 74. (2.1)

- Si un elemento queda con todos los menores a la izquierda y los mayores a la derecha, basta ordenar cada lado.
- Quicksort **fabrica** ese separador mediante partición del subarreglo.

De quicksortprelim a qksort

Subarreglos con límites

La versión preliminar falla porque no puede ordenar una porción interna.

De quicksortprelim a qksort

Subarreglos con límites

La versión preliminar falla porque no puede ordenar una porción interna. Solución: trabajar con $x[\text{left}], \dots, x[\text{right}]$.

De quicksortprelim a qksort

Subarreglos con límites

La versión preliminar falla porque no puede ordenar una porción interna. Solución: trabajar con $x[\text{left}], \dots, x[\text{right}]$.

Rutina final (esqueleto)

- `qksort( $x$ :array;  $left, right$ :int)`
 - si $right - left \geq 1$: `split( $x, left, right, i$ );`
`qksort( $x, left, i - 1$ ); qksort( $x, i + 1, right$ ).`
- `Quicksort( $x; n$ ):` llamar `qksort( $x, 1, n$ )`.

Partición `split` (Lomuto)

`split`($x, left, right, i$) — resumen

1. $L :=$ entero aleatorio en $[left, right]$; `swap`($x[left], x[L]$).

Partición `split` (Lomuto)

`split`($x, left, right, i$) — resumen

1. $L :=$ entero aleatorio en $[left, right]$; `swap`($x[left], x[L]$).
2. $T := x[left]$; $i := left$.

Partición `split` (Lomuto)

`split`($x, left, right, i$) — resumen

1. $L :=$ entero aleatorio en $[left, right]$; `swap`($x[left], x[L]$).
2. $T := x[left]$; $i := left$.
3. para $j := left + 1$ hasta $right$:
 - si $x[j] < T$: $i := i + 1$; `swap`($x[i], x[j]$).

Partición `split` (Lomuto)

`split`($x, left, right, i$) — resumen

1. $L :=$ entero aleatorio en $[left, right]$; `swap`($x[left], x[L]$).
2. $T := x[left]$; $i := left$.
3. para $j := left + 1$ hasta $right$:
 - si $x[j] < T$: $i := i + 1$; `swap`($x[i], x[j]$).
4. `swap`($x[left], x[i]$); ahora $x[i] = T$, menores a la izquierda, no menores a la derecha.

Partición `split` (Lomuto)

`split`($x, left, right, i$) — resumen

1. $L :=$ entero aleatorio en $[left, right]$; `swap`($x[left], x[L]$).
2. $T := x[left]$; $i := left$.
3. para $j := left + 1$ hasta $right$:
 - si $x[j] < T$: $i := i + 1$; `swap`($x[i], x[j]$).
4. `swap`($x[left], x[i]$); ahora $x[i] = T$, menores a la izquierda, no menores a la derecha.

Separador aleatorio

Elegir T al azar simplifica el análisis promedio y funciona bien en práctica.

Correctitud de `split`

Teorema 1

Teorema 2.1. *El procedimiento `split` divide correctamente el arreglo x alrededor del valor elegido T .*

Correctitud de `split`

Teorema 1

Teorema 2.1. *El procedimiento `split` divide correctamente el arreglo x alrededor del valor elegido T .*

Invariantes durante el bucle

Al avanzar j de $left + 1$ a $right$ se mantiene:

- (a) $x[left] = T$ y
- (b) $x[r] < T$ para todo $left < r \leq i$ y
- (c) $x[r] \geq T$ para todo $i < r \leq j$.

Peor caso de Quicksort

Escenario

- En cada llamada, el pivote resulta ser el **más pequeño**.

Peor caso de Quicksort

Escenario

- En cada llamada, el pivote resulta ser el **más pequeño**.
- Particionar cuesta $n - 1$ comparaciones; subproblemas de tamaños 0 y $n - 1$.

Peor caso de Quicksort

Escenario

- En cada llamada, el pivote resulta ser el **más pequeño**.
- Particionar cuesta $n - 1$ comparaciones; subproblemas de tamaños 0 y $n - 1$.

Recurrencia y solución

$$L(n) = L(n - 1) + n - 1 \quad (n \geq 1; L(0) = 0).$$

Peor caso de Quicksort

Escenario

- En cada llamada, el pivote resulta ser el **más pequeño**.
- Particionar cuesta $n - 1$ comparaciones; subproblemas de tamaños 0 y $n - 1$.

Recurrencia y solución

$$L(n) = L(n - 1) + n - 1 \quad (n \geq 1; L(0) = 0).$$

$$L(n) = 1 + 2 + \cdots + (n - 1) = \Theta(n^2).$$

- Modelo de ordenación por comparaciones e intercambios.

- Modelo de ordenación por comparaciones e intercambios.
- Quicksort: fabrica un separador y ordena recursivamente cada lado.

- Modelo de ordenación por comparaciones e intercambios.
- Quicksort: fabrica un separador y ordena recursivamente cada lado.
- Partición **split** (Lomuto) es **correcta** (Teorema 2.1).

- Modelo de ordenación por comparaciones e intercambios.
- Quicksort: fabrica un separador y ordena recursivamente cada lado.
- Partición **split** (Lomuto) es **correcta** (Teorema 2.1).
- Peor caso: $L(n) = \Theta(n^2)$; próxima clase: análisis de **caso promedio**.